

Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

Andreas Sommer, Lev Gromov, Marvin Hertweck, Pilar Keller

July 09, 2026

Task 1: Introductory Example

Consider this switched system on time horizon $T = [0, 30]$ with initial values $x_1(0) = 0, x_2(0) = 1000$ and parameter $p = (\frac{3}{4}, \frac{1}{4})$:

$$\frac{d}{dt}x_1(t) = \begin{cases} q_1(t), & \text{if } x_1 \leq 750 \\ q_1(t) - q_2(t), & \text{if } x_1(t) > 750 \end{cases}$$
$$\frac{d}{dt}x_2(t) = \begin{cases} 0, & \text{if } x_1 \leq 750 \\ -300, & \text{if } x_1(t) \in (750, 800) \\ 0 & \text{if } x_1(t) > 750 \end{cases}$$

where

$$q_1(t) = \frac{50}{27}t^2 - \frac{100}{3}t + \frac{400}{3}$$
$$q_2(t) = (1000 - x_2)(p_1(t - 19) + p_2)$$

- a) First, solve this system using `ode45` and look at the solution. To do so, define the right-hand side with the following function header: `function dx = rhsTask1(t, x, p)`. Implement the helper functions `q1` and `q2` as local variables. Does the solution look correct? Can you think of the reason why `ode45` produces the results it produces?
- b) Let's see how the solution responds to changes in the initial values: Plot the sensitivities of the solution using `plotSensitivities_ode45(sol, @rhsTask1, p, @ode45, options)`. Do they look correct?
- c) Now, solve this system using `IFDIFF`. What changed in comparison to a)? Use the same function header and implementation of the helper functions as mentioned in a)
- d) Plot the sensitivities of the solution with `IFDIFF` using `plotSensitivities_ifdiff()`. How do the sensitivities differ from the sensitivities you previously computed in Task 1?
- e) Let's take a look at a different method: Use the `MaxStep` parameter to `odeset` to produce a forward solution with small step sizes: `options = odeset(..., 'MaxStep', 0.1);`. Does it look correct?
- f) Now, compute and plot the sensitivities of this solution. What do you notice?

For reference, your solution and sensitivity plots with `IFDIFF` should look like the following:

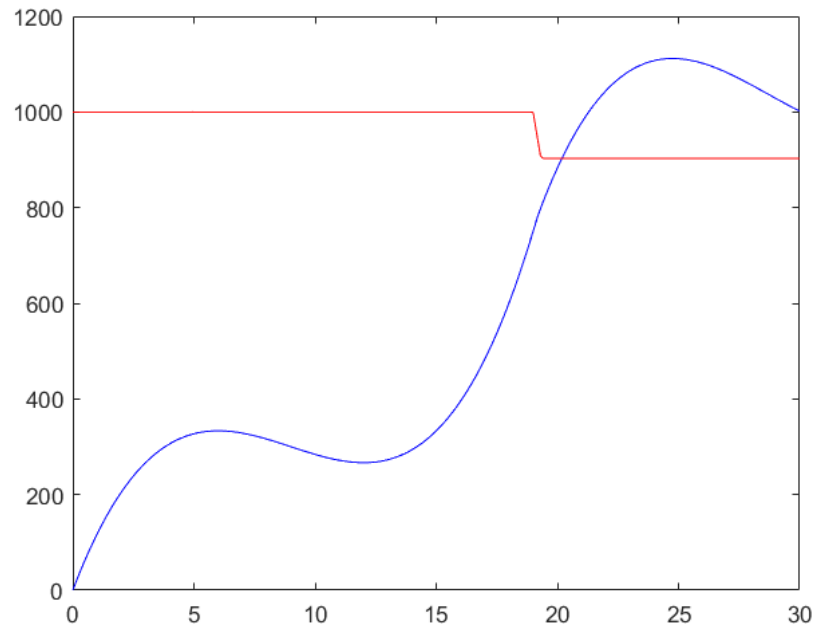


Figure 1: Solution plot for c)

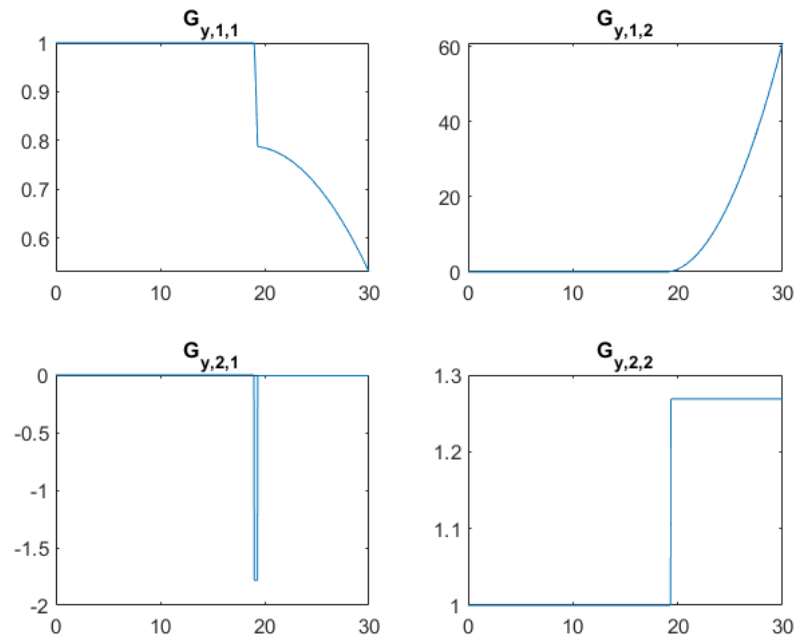


Figure 2: Sensitivities for d)

Appendix B: IFDIFF Cheat Sheet

RHS Function Requirements

- Signature: `function dx = myRHS(t, x, p)`. Keep in mind that the function file also needs to be named `myRHS.m`
- Helper functions work when defined either as:
 - in separate function files
 - as local variables
 - nested functions

but **DO NOT** work as local functions of `myRHS.m` !

- **Warning:** Do not use `elseif` statements, instead use nested `if ... else ... end` statements to code the switching behavior.

IFDIFF Solution Setup: A Canonical Example

We consider the following ODE:

$$\dot{x} = f(t, x, p) = \begin{pmatrix} f_1(t, x, p) \\ f_2(t, x, p) \end{pmatrix}$$
$$f_1(t, x, p) = 0.01 \cdot t^2 + x_2^3 \quad f_2(t, x, p) = \begin{cases} 0 & \text{if } x_1 < p \\ 5 & \text{if } p \leq x_1 < p + 0.5 \\ 0 & \text{if } x_1 \geq p + 0.5 \end{cases}$$

with the initial value $x(0) = (0, 1)^T$ and parameter $p = 5.437$

1. Setup the the RHS and name the file `canonicalExampleRHS.m`

```
function dx = canonicalExampleRHS(t,x,p)

dx = zeros(2,1);
dx(1) = 0.01 * t.^2 + x(2).^3;
if x(1) < p(1)
dx(2) = 0;
else
if x(1) < p(1) + 0.5
dx(2) = 5;
else
dx(2) = 0;
end
end

end
```

2. **Main Script:** Setup the initial values, the time interval as well as parameter(s):

```
t0 = 0;
tf = 20;
timeinterval = [t0,tf];
initstates   = [1 0];
p            = 5.437;
```

3. Define options in the main script and call `prepareDatahandleForIntegration` to prepare the right-hand side with the options for IFDIFF:

```
integrator = @ode45;
odeoptions = odeset( 'AbsTol', 1e-10, 'RelTol', 1e-5);
filename = 'canonicalExampleRHS';
datahandle = prepareDatahandleForIntegration(filename, 'integrator',
    integrator, 'options', odeoptions);
```

4. Create a solution structure using `solveODE` and evaluate

```
T = t0:0.1:tf;
sol_ifdiff = solveODE(datahandle, timeinterval, initstates, p);
Y_ifdiff = deval(sol_ifdiff, T);
```

5. Plot the solution

```
plot(T, Y_ifdiff);
xlabel('Time (s)');
ylabel('State Variables');
title('IFDIFF Solution');
grid on;
```

6. Compute and plot the sensitivities. There are two methods to do so. For the first one, you need to use `generateFDstep` so get finite differences and then use `generateSensitivityFunction` (which uses the VDE method in this example) to compute the sensitivities. Afterward, you can access and plot the sensitivities by appropriately reshaping the matrices in the `sensitivities` struct array, which are stored in the fields `Gy` for sensitivities w.r.t. initial values and `Gp` for sensitivities w.r.t. parameters respectively.

```
%% Step 5: Compute sensitivities manually

dim_y = size(Y_ifdiff, 1);
dim_p = length(p); % dim_p = 1

FDstep = generateFDstep(dim_y, dim_p, 'hy_rel_flag', true, 'hp_rel_flag', true, 'hy_min', 1e-6, 'hp_min', 1e-6);
sensFun = generateSensitivityFunction(datahandle, sol_ifdiff, FDstep, 'method', 'VDE', 'integrator_options', options);

sensitivities = sensFun(T); % Compute sensitivities at time points in T
```

Alternatively, you may use the provided `plotSensitivities_ifdiff` function which already computes the sensitivities internally and just outputs a plot:

```
plotSensitivities_ifdiff(datahandle, sol_ifdiff, p);
```

Your plots should look like the following:

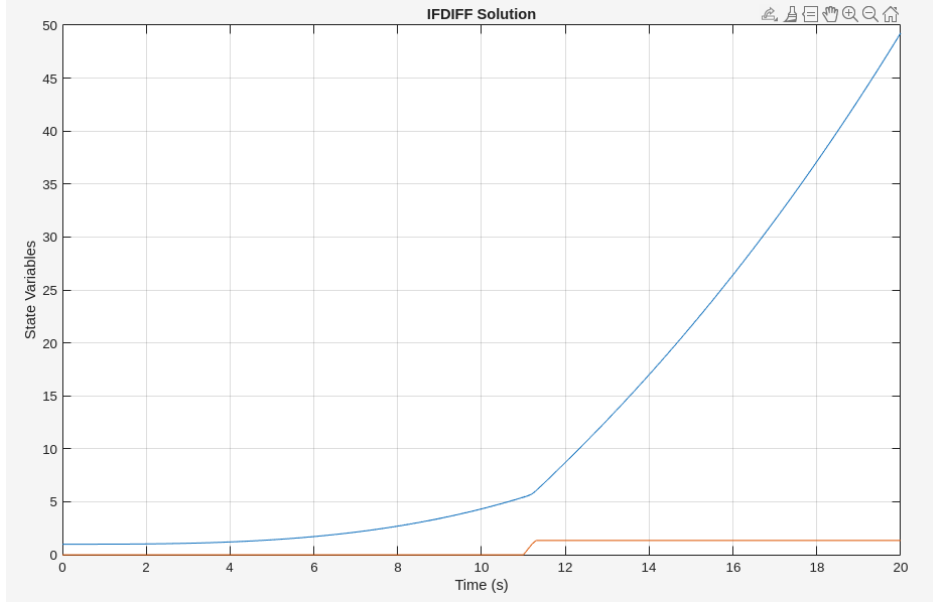


Figure 3: Canonical Example solution with IFDIFF

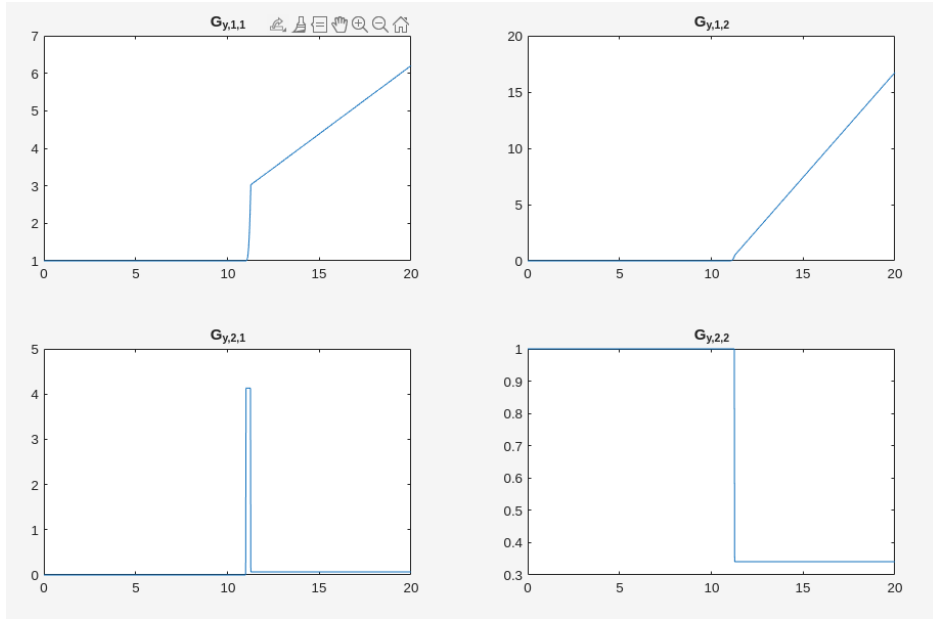


Figure 4: Sensitivities for the Canonical Example with IFDIFF

Specifying state jumps in a RHS

A state jump may be specified at any point in the RHS via a special if-block:

```
function dx = myRHS(t, x, p)
...
if ifdiff_jumpif(jumpCondition, jumpDirection)
... <statements computing the jumpIncrement> ...
endifdiff_update(jumpIncrement)
end
...
end
```

When the first argument of the `ifdiff_jumpif` function (`jumpCondition`) crosses zero in the direction specified by the second argument (`jumpDirection`), i.e. negative to positive (1), positive to negative (-1) or both (0), IFDIFF will increment the state vector in the solution by the first argument of the `ifdiff_update` function (`jumpIncrement`).

Warning: The state vector and `jumpIncrement` must have matching dimensions. In particular, the `jumpIncrement` must be a column-vector with the same number of elements as the state vector.

Warning: IFDIFF will never execute the body of the if-block directly in the RHS, so make sure that the statements in the if-block are only used to compute the `jumpIncrement`. In particular, any variables exclusively defined in the if-block must not be used outside of the if-block.

Plotting state jump sensitivities

Solve the system, compute the sensitivities and then call the `plotSensitivitiesSwitched` function.

Here is an example:

```
% RHS params
tspan = [0, 10];
h0 = 0;
v0 = 10;
x0 = [h0; v0];
p = 9.8;
% Solver params
solver = @ode45;
options = odeset('AbsTol', 1e-8, 'RelTol', 1e-6);
% Call IFDIFF
dh = prepareDatahandleForIntegration(@bounceRHS, 'integrator', solver, 'options', options);
sol = solveODE(dh, tspan, x0, p);

dim_y = size(sol.y, 1);
dim_p = length(p);
FDstep = generateFDstep(dim_y, dim_p);
sensFun = generateSensitivityFunction(dh, sol, FDstep, 'method', 'VDE');

plotSensitivitiesSwitched(sol, sensFun, tspan);
```